JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.zip

Class ZipFile

java.lang.Object
    java.util.zip.ZipFile

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Direct Known Subclasses:

`JarFile`

```
public class ZipFile
extends Object
implements Closeable
```

This class is used to read entries from a zip file.

Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

Field Summary

Fields

Modifier and Type	Field	Description
static int	**CENATT**	
static int	**CENATX**	
static int	**CENCOM**	
static int	**CENCRC**	
static int	**CENDSK**	

static int	**CENEXT**	
static int	**CENFLG**	
static int	**CENHDR**	
static int	**CENHOW**	
static int	**CENLEN**	
static int	**CENNAM**	
static int	**CENOFF**	
static long	**CENSIG**	
static int	**CENSIZ**	
static int	**CENTIM**	
static int	**CENVEM**	
static int	**CENVER**	
static int	**ENDCOM**	
static int	**ENDHDR**	
static int	**ENDOFF**	
static long	**ENDSIG**	
static int	**ENDSIZ**	
static int	**ENDSUB**	
static int	**ENDTOT**	
static int	**EXTCRC**	
static int	**EXTHDR**	
static int	**EXTLEN**	
static long	**EXTSIG**	
static int	**EXTSIZ**	
static int	**LOCCRC**	
static int	**LOCEXT**	
static int	**LOCFLG**	
static int	**LOCHDR**	
static int	**LOCHOW**	

static int	LOCLEN	
static int	LOCNAM	
static long	LOCSIG	
static int	LOCSIZ	
static int	LOCTIM	
static int	LOCVER	
static int	OPEN_DELETE	Mode flag to open a zip file and mark it for deletion.
static int	OPEN_READ	Mode flag to open a zip file for reading.

Constructor Summary

Constructors

Constructor	Description
ZipFile(File file)	Opens a ZIP file for reading given the specified File object.
ZipFile(File file, Charset charset)	Opens a ZIP file for reading given the specified File object.
ZipFile(File file, int mode)	Opens a new ZipFile to read from the specified File object in the specified mode.
ZipFile(File file, int mode, Charset charset)	Opens a new ZipFile to read from the specified File object in the specified mode.
ZipFile(String name)	Opens a zip file for reading.

`ZipFile(String name, Charset charset)`	Opens a zip file for reading.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`void`	`close()`	Closes the ZIP file.
`Enumeration<? extends ZipEntry>`	`entries()`	Returns an enumeration of the ZIP file entries.
`protected void`	`finalize()`	Ensures that the system resources held by this ZipFile object are released when there are no more references to it.
`String`	`getComment()`	Returns the zip file comment, or null if none.
`ZipEntry`	`getEntry(String name)`	Returns the zip file entry for the specified name, or null if not found.
`InputStream`	`getInputStream(ZipEntry entry)`	Returns an input stream for reading the contents of

		the specified zip file entry.
String	getName()	Returns the path name of the ZIP file.
int	size()	Returns the number of entries in the ZIP file.
Stream<? extends ZipEntry>	stream()	Return an ordered Stream over the ZIP file entries.

Methods inherited from class java.lang.Object

clone, equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

OPEN_READ

```
public static final int OPEN_READ
```

Mode flag to open a zip file for reading.

See Also:

Constant Field Values

OPEN_DELETE

```
public static final int OPEN_DELETE
```

Mode flag to open a zip file and mark it for deletion. The file will be deleted some time between the moment that it is opened and the moment that it is closed, but its contents will remain accessible via the ZipFile object until either the close method is invoked or the virtual machine exits.

See Also:

Constant Field Values

LOCSIG

public static final long LOCSIG

See Also:

Constant Field Values

EXTSIG

public static final long EXTSIG

See Also:

Constant Field Values

CENSIG

public static final long CENSIG

See Also:

Constant Field Values

ENDSIG

public static final long ENDSIG

See Also:

Constant Field Values

LOCHDR

public static final int LOCHDR

See Also:

Constant Field Values

EXTHDR

public static final int EXTHDR

See Also:

Constant Field Values

CENHDR

public static final int CENHDR

See Also:

Constant Field Values

ENDHDR

public static final int ENDHDR

See Also:

Constant Field Values

LOCVER

public static final int LOCVER

See Also:

Constant Field Values

LOCFLG

public static final int LOCFLG

See Also:

Constant Field Values

LOCHOW

public static final int LOCHOW

See Also:

Constant Field Values

LOCTIM

```
public static final int LOCTIM
```

See Also:

Constant Field Values

LOCCRC

```
public static final int LOCCRC
```

See Also:

Constant Field Values

LOCSIZ

```
public static final int LOCSIZ
```

See Also:

Constant Field Values

LOCLEN

```
public static final int LOCLEN
```

See Also:

Constant Field Values

LOCNAM

```
public static final int LOCNAM
```

See Also:

Constant Field Values

LOCEXT

```
public static final int LOCEXT
```

See Also:

Constant Field Values

EXTCRC

public static final int EXTCRC

See Also:

Constant Field Values

EXTSIZ

public static final int EXTSIZ

See Also:

Constant Field Values

EXTLEN

public static final int EXTLEN

See Also:

Constant Field Values

CENVEM

public static final int CENVEM

See Also:

Constant Field Values

CENVER

public static final int CENVER

See Also:

Constant Field Values

CENFLG

public static final int CENFLG

See Also:

Constant Field Values

CENHOW

public static final int CENHOW

See Also:

Constant Field Values

CENTIM

public static final int CENTIM

See Also:

Constant Field Values

CENCRC

public static final int CENCRC

See Also:

Constant Field Values

CENSIZ

public static final int CENSIZ

See Also:

Constant Field Values

CENLEN

public static final int CENLEN

See Also:

Constant Field Values

CENNAM

public static final int CENNAM

See Also:

Constant Field Values

CENEXT

public static final int CENEXT

See Also:

Constant Field Values

CENCOM

public static final int CENCOM

See Also:

Constant Field Values

CENDSK

public static final int CENDSK

See Also:

Constant Field Values

CENATT

public static final int CENATT

See Also:

Constant Field Values

CENATX

public static final int CENATX

See Also:

Constant Field Values

CENOFF

public static final int CENOFF

See Also:

Constant Field Values

ENDSUB

public static final int ENDSUB

See Also:

Constant Field Values

ENDTOT

public static final int ENDTOT

See Also:

Constant Field Values

ENDSIZ

public static final int ENDSIZ

See Also:

Constant Field Values

ENDOFF

public static final int ENDOFF

See Also:

Constant Field Values

ENDCOM

public static final int ENDCOM

See Also:

Constant Field Values

Constructor Detail

ZipFile

```
public ZipFile(String name)
        throws IOException
```

Opens a zip file for reading.

First, if there is a security manager, its `checkRead` method is called with the `name` argument as its argument to ensure the read is allowed.

The UTF-8 `charset` is used to decode the entry names and comments.

Parameters:

`name - the name of the zip file`

Throws:

`ZipException - if a ZIP format error has occurred`

`IOException - if an I/O error has occurred`

`SecurityException - if a security manager exists and its checkRead method doesn't allow read access to the file.`

See Also:

`SecurityManager.checkRead(java.lang.String)`

ZipFile

```
public ZipFile(File file,
               int mode)
        throws IOException
```

Opens a new `ZipFile` to read from the specified `File` object in the specified mode. The mode argument must be either `OPEN_READ` or `OPEN_READ | OPEN_DELETE`.

First, if there is a security manager, its `checkRead` method is called with the `name` argument as its argument to ensure the read is allowed.

The UTF-8 `charset` is used to decode the entry names and comments

Parameters:

`file - the ZIP file to be opened for reading`

`mode - the mode in which the file is to be opened`

Throws:

`ZipException - if a ZIP format error has occurred`

`IOException - if an I/O error has occurred`

SecurityException - if a security manager exists and its checkRead method doesn't allow read access to the file, or its checkDelete method doesn't allow deleting the file when the OPEN_DELETE flag is set.

IllegalArgumentException - if the mode argument is invalid

Since:

1.3

See Also:

SecurityManager.checkRead(java.lang.String)

ZipFile

```
public ZipFile(File file)
        throws ZipException,
               IOException
```

Opens a ZIP file for reading given the specified File object.

The UTF-8 charset is used to decode the entry names and comments.

Parameters:

file - the ZIP file to be opened for reading

Throws:

ZipException - if a ZIP format error has occurred

IOException - if an I/O error has occurred

ZipFile

```
public ZipFile(File file,
               int mode,
               Charset charset)
        throws IOException
```

Opens a new ZipFile to read from the specified File object in the specified mode. The mode argument must be either OPEN_READ or OPEN_READ | OPEN_DELETE.

First, if there is a security manager, its checkRead method is called with the name argument as its argument to ensure the read is allowed.

Parameters:

file - the ZIP file to be opened for reading

mode - the mode in which the file is to be opened

charset - the charset to be used to decode the ZIP entry name and comment that are not encoded by using UTF-8 encoding (indicated by entry's general purpose flag).

Throws:

ZipException - if a ZIP format error has occurred

IOException - if an I/O error has occurred

SecurityException - if a security manager exists and its checkRead method doesn't allow read access to the file,or its checkDelete method doesn't allow deleting the file when the OPEN_DELETE flag is set

IllegalArgumentException - if the mode argument is invalid

Since:

1.7

See Also:

SecurityManager.checkRead(java.lang.String)

ZipFile

```
public ZipFile(String name,
               Charset charset)
        throws IOException
```

Opens a zip file for reading.

First, if there is a security manager, its checkRead method is called with the name argument as its argument to ensure the read is allowed.

Parameters:

name - the name of the zip file

charset - the charset to be used to decode the ZIP entry name and comment that are not encoded by using UTF-8 encoding (indicated by entry's general purpose flag).

Throws:

ZipException - if a ZIP format error has occurred

IOException - if an I/O error has occurred

SecurityException - if a security manager exists and its checkRead method doesn't allow read access to the file

Since:

1.7

See Also:

SecurityManager.checkRead(java.lang.String)

ZipFile

```
public ZipFile(File file,
               Charset charset)
        throws IOException
```

Opens a ZIP file for reading given the specified File object.

Parameters:

`file - the ZIP file to be opened for reading`

`charset - The charset to be used to decode the ZIP entry name and comment (ignored if the language encoding bit of the ZIP entry's general purpose bit flag is set).`

Throws:

`ZipException - if a ZIP format error has occurred`

`IOException - if an I/O error has occurred`

Since:

`1.7`

Method Detail

getComment

```
public String getComment()
```

Returns the zip file comment, or null if none.

Returns:

`the comment string for the zip file, or null if none`

Throws:

`IllegalStateException - if the zip file has been closed Since 1.7`

getEntry

```
public ZipEntry getEntry(String name)
```

Returns the zip file entry for the specified name, or null if not found.

Parameters:

`name - the name of the entry`

Returns:

`the zip file entry, or null if not found`

Throws:

`IllegalStateException` - if the zip file has been closed

getInputStream

```
public InputStream getInputStream(ZipEntry entry)
                           throws IOException
```

Returns an input stream for reading the contents of the specified zip file entry.

Closing this ZIP file will, in turn, close all input streams that have been returned by invocations of this method.

Parameters:

`entry` - the zip file entry

Returns:

the input stream for reading the contents of the specified zip file entry.

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

`IllegalStateException` - if the zip file has been closed

getName

```
public String getName()
```

Returns the path name of the ZIP file.

Returns:

the path name of the ZIP file

entries

```
public Enumeration<? extends ZipEntry> entries()
```

Returns an enumeration of the ZIP file entries.

Returns:

an enumeration of the ZIP file entries

Throws:

`IllegalStateException` - if the zip file has been closed

stream

```
public Stream<? extends ZipEntry> stream()
```

Return an ordered `Stream` over the ZIP file entries. Entries appear in the `Stream` in the order they appear in the central directory of the ZIP file.

Returns:

an ordered Stream of entries in this ZIP file

Throws:

`IllegalStateException` - if the zip file has been closed

Since:

1.8

size

```
public int size()
```

Returns the number of entries in the ZIP file.

Returns:

the number of entries in the ZIP file

Throws:

`IllegalStateException` - if the zip file has been closed

close

```
public void close()
           throws IOException
```

Closes the ZIP file.

Closing this ZIP file will close all of the input streams previously returned by invocations of the `getInputStream` method.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Throws:

`IOException` - if an I/O error has occurred

finalize

```
protected void finalize()
                 throws IOException
```

Ensures that the system resources held by this ZipFile object are released when there are no more references to it.

Since the time when GC would invoke this method is undetermined, it is strongly recommended that applications invoke the `close` method as soon they have finished accessing this `ZipFile`. This will prevent holding up system resources for an undetermined length of time.

Overrides:

`finalize` in class `Object`

Throws:

`IOException` - if an I/O error has occurred

See Also:

`close()`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.